

Umoja Voices

Choir Management App — As-Built Reference (updated 2026-07-19, v0.13.3)

This document originally laid out the proposed design for a choir management web app. It has since been built, deployed on Vercel with a real Supabase Postgres database, and iterated through many minor releases. This revision updates the as-built reference (last revised at v0.12.1) to cover the v0.13.x Songs/Media/Lyrics UI rebuild, driven by a second round of app-owner-supplied mockups (umoja-update.pdf), and calls out what remains intentionally deferred. Where a feature described here isn't built yet, it's marked (planned, not yet built) with its tracking issue number.

1. Roles & Access

Two roles cover everything described: Admin and Chorister. There is no public/anonymous access to songs or member data — everything except the external links page requires login.

Capability	Chorister	Admin
View song list & song detail (media + lyrics)	Yes	Yes
Add / edit / delete songs, voice-part sections, media, lyrics	No	Yes
View external links page	Yes	Yes
Manage external links (add / remove)	No	Yes
View logistics (deadlines, itinerary, practice schedule)	Yes	Yes
Manage logistics (create trip; add / remove deadlines, itinerary, practice sessions)	No	Yes
Invite new members	No	Yes
View / edit member list, change roles, deactivate, delete	No	Yes
Edit own profile (name, password)	Yes	Yes

Invite flow: an admin enters a new member's name and email on the Members screen. The system creates a pending account and an Invite record with a secure, expiring token, and returns the invite link directly on screen for the admin to share manually — it is not yet emailed automatically. The invitee opens the link, sets a password, and lands in the chorister view already logged in. There is no public self-signup — accounts only exist via admin invite, which keeps membership limited to actual choir members. Wiring this through a transactional email provider (Resend) so the link is emailed instead of hand-copied is tracked as remaining work (issue #9).

Forgot password: a member who forgets their password doesn't need an admin's help to get back in. From the login screen they follow a Forgot password? link, enter their email, and — if it matches an active account — get a self-service reset email with a secure, expiring link to set a

new password. This is implemented today with the app's own PasswordResetToken table and NextAuth Credentials sessions, timing-safe against email enumeration — not yet Supabase Auth's built-in reset flow. Migrating the auth layer itself from NextAuth Credentials to Supabase Auth (the database already runs on Supabase Postgres) is tracked as remaining work (issue #10).

2. Data Model

A relational schema (PostgreSQL via Supabase, accessed through Prisma) — the tables below are the actual, currently-deployed models (prisma/schema.prisma), named to match their Prisma model names.

User

Column	Type	Notes
id	text (cuid)	primary key
email	text, unique	login identifier
name	text	display name
role	"admin" / "chorister"	
status	"invited" / "active" / "disabled"	
passwordHash	text, nullable	set at invite acceptance
mustChangePassword	boolean	forces a password change on next login for known/default passwords (e.g. seeded accounts)
calendarToken	text, unique, nullable	reserved for the planned per-member ICS feed (issue #11) — not yet consumed by any route
invitedById	text, fk -> User.id, nullable	
createdAt / updatedAt	timestamp	

Invite

Column	Type	Notes
id	text (cuid)	primary key
email	text	
token	text, unique	random, expiring
role	"admin" / "chorister"	role granted on acceptance
expiresAt	timestamp	7 days
acceptedAt	timestamp, nullable	
invitedById	text, fk -> User.id	
createdAt	timestamp	

PasswordResetToken

Column	Type	Notes
id	text (cuid)	primary key
token	text, unique	random, expiring
userId	text, fk -> User.id	
expiresAt	timestamp	
usedAt	timestamp, nullable	
createdAt	timestamp	

Song

Carries the title plus optional composer/lyricist credit fields (added in v0.13.0, both nullable) — everything about how it's sung lives on SongSection below.

Column	Type	Notes
id	text (cuid)	primary key
title	text	
composer	text, nullable	shown as the "Music:" credit on the Songs list and song/media/lyrics pages when present
lyricist	text, nullable	shown as the "Words:" credit alongside composer when present
createdById	text, fk -> User.id	
createdAt / updatedAt	timestamp	

SongSection

One row per named voice-part grouping on a song (e.g. "Tenor", "SATB"). Replaces the original flat song_parts table.

Column	Type	Notes
id	text (cuid)	primary key
songId	text, fk -> Song.id	
part	"S" / "A" / "T" / "B" / "All"	drives the section's identity/badge; "All" displays as "SATB" on the Media page (Section 3) — a deliberately different label from the Lyrics page's "ALL" (Section 4), from two separate rounds of live feedback
sectionLabel	text	e.g. "Tenor"

Column	Type	Notes
labelDescription	text	free-text description for this section
sortOrder	int	

SongMedia

One or more media items nested under a SongSection — this is what actually lets "Tenor" group both an audio take and a video take together.

Column	Type	Notes
id	text (cuid)	primary key
sectionId	text, fk -> SongSection.id	
label	text	e.g. "Tenor video"
mediaUrl	text	pasted external link or direct file URL
mediaKind	"audio" / "video" / "youtube" / "drive" / "soundcloud" / "direct_url"	auto-detected from the pasted URL; drives which embed player renders
sortOrder	int	

LyricSection

Column	Type	Notes
id	text (cuid)	primary key
songId	text, fk -> Song.id	
sectionType	"verse" / "chorus" / "bridge" / "intro" / "outro" / "vamp" / "custom"	
sectionLabel	text	e.g. "Verse 2" — auto-suggested from the pasted label line, editable
content	text	the actual lyric lines
voiceTags	text, comma-separated, default "SATB"	which voice(s) sing this section; displayed as "ALL" wherever "SATB" would otherwise show
sortOrder	int	

ExternalLink

Column	Type	Notes
id	text (cuid)	primary key
title	text	e.g. "Instagram"

Column	Type	Notes
url	text	
category	"social" / "news" / "media" / "other"	groups the links page into sections
sortOrder	int	

Trip

Column	Type	Notes
id	text (cuid)	primary key
title	text	e.g. "Portugal & Germany Tour 2026"
destination	text	
startDate / endDate	date	
status	"upcoming" / "past"	
createdById	text, fk -> User.id	
createdAt	timestamp	

LogisticsDeadline

Column	Type	Notes
id	text (cuid)	primary key
tripId	text, fk -> Trip.id	
category	"visa" / "tickets" / "payment" / "other"	
label	text	e.g. "Visa application deadline"
dueDate	date	
notes	text, nullable	
sortOrder	int	

ItineraryItem

Column	Type	Notes
id	text (cuid)	primary key
tripId	text, fk -> Trip.id	
date	date	
time	text, nullable	
title	text	e.g. "Depart Nairobi", "Joint concert"

Column	Type	Notes
location	text, nullable	
notes	text, nullable	
sortOrder	int	

PracticeSession

Column	Type	Notes
id	text (cuid)	primary key
tripId	text, fk -> Trip.id, nullable	nullable — supports general rehearsals not tied to a trip
date	date	
time	text	
location	text	
notes	text, nullable	
sortOrder	int	

3. Songs: Sections, Media & Embedded Players

Creating or editing a song's full structure (title, credits, voice-part sections, media) is a form reached via the song's Edit button:

- Title, and optional Words (lyricist) / Music (composer) credit fields.
- Voice-part sections — a repeatable list, each with a part picker (S / A / T / B / All), a free-text section label (e.g. "Tenor"), and a label description. Each section then holds its own repeatable list of media items (label + pasted URL), so one section can group several recordings — for example a "Tenor" section containing both an audio take and a video take, each independently addable/removable.

Pasted link — YouTube, Google Drive share link, SoundCloud, or a direct .mp3/.mp4 URL. The app detects the link type automatically and stores the right mediaKind.

Direct URL (fallback) — if the pasted link doesn't match a recognized provider or file type, it's stored as a plain `direct_url` rather than rejected. The media item still gets saved and shown, just as an "Open link" button instead of an inline embed.

Direct upload (recording a part straight into the app's own storage instead of pasting a link) remains deliberately out of scope — the app works from pasted links only, per Section 11. Supabase Storage is still the right tool for it later; it just isn't wired up.

Browsing songs: /songs is a plain grid of song title cards — no filter or search — each showing the Words/Music credit line underneath the title when either is set. Admins see an "Add Song" button in the top-right of this list; choristers don't.

Song detail (/songs/[id]): shows the title, credits, and admin-only Edit/Delete buttons, plus two clickable summary cards — Media and Lyrics — each linking to its own page (added in v0.13.0;

previously both were rendered inline on this page).

Media page (/songs/[id]/media, added in v0.13.0): every recording grouped under its voice part — Soprano, Alto, Tenor, Bass, and SATB (the "All" part's display label on this page specifically, changed from an earlier "Tutti (full choir)" wording in v0.13.2 per the app owner's direct request — the Lyrics page's equivalent "everyone" state uses the word "ALL" instead; see Section 4). Only voice groups that actually have media are shown — an empty group is invisible, not a "No media yet." placeholder (v0.13.3). Each media item renders with its matching embed player (see below) and, for admins, a Remove button. Admins additionally get a quick Add Audio form directly on this page — URL, Label, and Voice are all required — that appends the new item to the right voice group's section (creating one if it doesn't already exist) without needing to open the full song editor. The full editor (title/credits/voice-part sections with labels and descriptions) remains available via the song's Edit button for structural changes.

On the media page and the (legacy, still-supported) full editor's preview, every media item renders with an inline embedded player matched to its kind — a YouTube iframe embed, a Google Drive preview iframe, a native HTML5 <audio>/<video> element for direct files, or a plain "Open link" button for the `direct_url` fallback — so playback happens without leaving the app.

4. Structured Lyrics

Lyrics are entered and edited as a series of labeled sections rather than a single free-text box, so the structure (verse/chorus/bridge) is a real, editable property. Two ways to add/replace lyrics coexist:

Replace Lyrics (paste-and-parse, added v0.13.0) — reached via an admin-only Edit button on the Lyrics page (/songs/[id]/lyrics/edit). An admin pastes raw lyrics into a textarea: blank-line-separated segments, where each segment's first line is its label (e.g. "Verse 1", "Chorus") and subsequent lines may start with a voice tag — [S], [A], [T], [B], or [ALL]. A tag carries forward across subsequent segments until a new tag appears, so a single [A] before an untagged run of segments tags all of them Alto until the next explicit tag. A Preview button parses the pasted text and shows the resulting segments (label, resolved voice tag, lines) before anything is saved. Save commits for a song with no existing lyrics; Replace Lyrics (destructive-styled, confirms before overwriting) replaces existing lyrics; Cancel discards and returns to the Lyrics page. An Upload tab is present alongside Paste as a disabled placeholder — the infrastructure for a future file-import feature, not yet wired up (planned, not yet built).

Manual section editor (original, still available) — inside the full song editor (/admin/songs/[id]/edit), an admin can instead add sections one at a time — pick a type from a dropdown (Verse, Chorus, Bridge, Intro, Outro, Vamp, Custom), optionally override the label (defaults to the type name), type the lyric text directly, and tick which voice(s) sing it (S / A / T / B, defaulting to SATB). Sections can be removed individually. This path has no tag-carry-forward parsing — each section's voice tags are set explicitly via checkbox — and is useful for fine-tuning a single section without re-pasting the whole song.

A drag-and-drop section reorder (order currently follows entry order only) and a true per-line voice-tag data model (as opposed to today's one-tag-per-section) were both considered and are not currently on the roadmap — every worked example reviewed with the app owner resolved to one consistent tag per section, so the simpler section-level model was kept rather than adding a LyricLine schema migration for it.

Display to choristers: the Lyrics page (/songs/[id]/lyrics) renders each section under its label, with its voice tag(s) shown as small badges (S / A / T / B / ALL — "SATB" always displays as "ALL", reverted from a brief "SATB" rename in v0.13.0 after direct feedback once it shipped).

Filtering lyrics by voice: the page shows the current view's tag top-left (defaulting to ALL) and clickable S / A / T / B filter tabs. Clicking one narrows the view to just the sections tagged for that voice plus any ALL (full-choir) sections; clicking the active tab again returns to the unfiltered ALL view. This is implemented and working as designed.

5. External Links Page

A simple, admin-managed page grouping links by category — Social Media, News, Media/Press, Other — each shown as a title with an outbound link. Visible to any logged-in member. Admins see an inline "Edit" button on /links itself that opens /admin/links, where links can be added or removed (there is no edit-in-place or manual reordering of existing links yet — only create and delete).

6. Logistics & Travel

Since the choir travels to perform, members need one place to see the practical details of an upcoming trip. A logistics page is built around the current (or next) trip and has three parts: key dates/deadlines, itinerary, and practice schedule — all admin-editable, chorister read-only. The seeded first real trip is the Portugal & Germany tour, 7–18 October 2026.

Choristers see all three sections read-only at /logistics. Admins additionally see an inline "Edit" button that opens /admin/logistics, where a trip can be created (if none exists yet) and deadlines/itinerary items/practice sessions can be added or removed (again, no in-place editing of an existing row yet — only add and delete).

Calendar sync & notifications (planned, not yet built — issue #11). The original design called for a personal ICS subscription feed per member (User.calendarToken already exists in the schema for this) plus email notifications with a one-click .ics attachment whenever a practice session or itinerary item changes. Neither the /api/calendar/[token].ics route nor the notification email exists yet — this is the next logistics feature queued up, and depends on the Resend wiring from Section 1 landing first.

7. Navigation & Route Map

There is no separate "Admin" section of the app anymore. A single shared Nav renders for every logged-in user; admin-only actions are surfaced inline, on the same pages choristers already use, rather than behind a dedicated admin tab or dashboard. The only admin-specific nav item is a "Members" tab (visible only to admins), since there's no chorister-facing member directory to attach it to.

Route	Access	Purpose
/login	public	Sign in; invite-acceptance and password-reset flows both land here after completing
/accept-invite/[token]	public (valid token)	Set password, activate account

Route	Access	Purpose
/forgot-password	public	Enter email to request a reset link
/reset-password/[token]	public (valid token)	Set a new password
/change-password	chorister + admin	Forced on first login for seeded/default-password accounts
/songs	chorister + admin	Browse songs; admins additionally see an "Add Song" button
/songs/[id]	chorister + admin	Title, credits, Media/Lyrics summary cards; admins additionally see Edit + Delete
/songs/[id]/media	chorister + admin	Media grouped by voice part; admins additionally see Remove and an "Add Audio" form
/songs/[id]/lyrics	chorister + admin	Filterable structured lyrics view; admins additionally see an Edit button
/songs/[id]/lyrics/edit	admin only	Paste-and-parse Replace Lyrics flow
/links	chorister + admin	External links page; admins additionally see an inline Edit button -> /admin/links
/logistics	chorister + admin	Current trip: key dates, itinerary, practice schedule (read-only); admins additionally see an inline Edit button -> /admin/logistics
/profile	chorister + admin	Edit own name, password
/admin/songs/new, /admin/songs/[id]/edit	admin only	The full song editor (title, credits, sections, media, lyrics) — reached via Edit/Add Song buttons above, not via nav
/admin/links	admin only	Add/remove external links — reached via the inline Edit button on /links
/admin/logistics	admin only	Create trip; add/remove deadlines, itinerary, practice sessions — reached via the inline Edit button on /logistics
/admin/members	admin only	Member list, invite form, role/status changes, deletion — reached via the "Members" nav tab
/api/calendar/[token].ics	—	Planned, not yet built (issue #11)

8. Tech Stack (as implemented)

Layer	Choice	Notes
Framework	Next.js 16 (App Router) + TypeScript	One codebase for pages + server actions; deployed natively to Vercel
Database	PostgreSQL via Supabase, provisioned through Vercel's Supabase Marketplace integration	Live production database, not a local/dev stand-in
ORM / data access	Prisma 7 (@prisma/adapters-pg)	Enum-like fields are plain String columns (not native Postgres enums), validated at the application layer (src/lib/validation.ts) rather than with Zod, despite Zod being an installed-but-unused dependency
Auth	NextAuth v4, Credentials provider, JWT sessions	A stand-in for Supabase Auth — migrating the auth layer itself is tracked as remaining work (issue #10); the database half of that migration is already done
File storage	Supabase Storage — deferred, not wired up	Pasted links only for now; see Section 3
Transactional email	Resend — planned, not yet wired up (issue #9)	Invite links are currently shown on-screen instead of emailed
Styling / UI	Tailwind CSS v4	Hand-rolled components; no component/UI kit (e.g. shadcn/ui) in use
Rate limiting	In-memory, per-process (src/lib/rate-limit.ts)	Covers login, forgot-password, and invite creation; needs a distributed store before multi-instance/serverless scaling (issue #20)
Security hardening	HTTP security headers + CSP, admin self-lockout guard, timing-safe password reset, server-side enum validation	Delivered in the initial security-audit pass; CSP still relies on 'unsafe-inline' — moving to a nonce-based CSP is tracked (issue #17)
Vercel platform config	.vercelignore anchored to the repo root (/Media/, /extras/)	Fixed in v0.13.1 after an unanchored Media/ pattern silently dropped the new songs/[id]/media route from every deploy

9. Hosting (live, not just recommended)

The app is deployed and live at <https://umoja-voices.vercel.app> — Vercel project `egm2/umoja-voices`, GitHub-connected so every push to main auto-deploys. The database is Supabase Postgres, connected via Vercel's Supabase Marketplace integration rather than hand-typed connection strings. Both platforms' free tiers comfortably cover the confirmed scale (Section 11) — no paid upgrade has been needed to run in production.

Render, Netlify, and self-hosting were considered during the original design pass and are still reasonable alternatives in the abstract, but are no longer live options under evaluation — the app already runs on Vercel + Supabase.

10. Delivered Scope vs. Remaining Work

Delivered (v0.1.0 through v0.13.3):

- Login, admin-driven invite flow, and self-service forgot-password, all backed by NextAuth Credentials + a real Supabase Postgres database.
- Forced password change on first login for seeded/default-password accounts.
- Full song management: title, optional Words/Music credits, voice-part sections each grouping multiple media items, and a structured, per-voice-tagged lyrics editor — with inline embedded playback for YouTube/Drive/SoundCloud/direct files.
- Browsing: a plain song list with credit subtitles, a song detail page with Media/Lyrics summary cards linking to their own pages.
- Media page grouped by voice part (empty groups hidden) with a quick Add Audio form.
- Lyrics page with a filterable, SATB/voice-tagged view, plus a paste-and-parse Replace Lyrics flow whose voice-tag carries forward across sections until a new explicit tag appears.
- External links page, admin-manageable (add/remove).
- Logistics page for one active trip (deadlines, itinerary, practice schedule), admin-manageable (add/remove).
- Member management: invite, role changes, activate/deactivate, and real deletion (guarded against self-delete and removing the last active admin, with a friendly error if the member has songs/trips/invites tied to their account).
- No separate admin area: admin actions surfaced inline per-section, plus an admin-only "Members" nav tab.
- Security hardening: HTTP security headers/CSP, self-lockout guard, timing-safe password reset, server-side enum validation, in-memory rate limiting on login/forgot-password/invite creation.
- Deployed to production on Vercel, GitHub-connected for auto-deploy, backed by a real Supabase Postgres database.

Remaining, tracked as GitHub issues:

- #9 — Wire up Resend for invite/password-reset emails (currently just displays the link on-screen).
- #10 — Migrate NextAuth Credentials to Supabase Auth (the database half is done; this is the auth-layer half).
- #11 — ICS calendar feed + email notifications for practice sessions/itinerary.
- #17 — Nonce-based CSP (remove 'unsafe-inline' from script-src/style-src).
- #18 — Rate-limit timing/delay tuning.
- #20 — Distributed rate-limit store (needed before multi-instance/serverless scaling).

Deliberately still out of scope, not currently tracked: bulk song import, a native mobile app, recurring-rule practice scheduling, multi-trip history, audit history, and a per-line lyrics voice-tag data model (every worked paste example resolved to one consistent tag per section, so this wasn't needed).

11. Confirmed Decisions

- Media: pasted external links only (YouTube, Google Drive, SoundCloud, direct file URLs, or the `direct_url` fallback) — no file-upload UI or storage bucket wiring. Direct upload remains a documented fast-follow, not something designed out.
- Branding: implemented — dark navbar, cream background, gold-accented cards, pulled from the existing @umojavoices Instagram profile (<https://www.instagram.com/umojavoices/>) rather than invented from scratch.
- Scale: ~25 members and ~20 songs. Comfortably within the free tiers of both Vercel and Supabase in production.
- First real trip: the Portugal/Germany performance trip, 7–18 October 2026, seeded as the first real Trip row (destination, key dates, itinerary, practice schedule) instead of placeholder data.
- Lyrics import format: paste-only for now — segments separated by a blank line, first line of each segment is its label, and a bracket voice tag ([S]/[A]/[T]/[B]/[ALL]) carries forward across segments until a new one appears. Confirmed against the app owner's own worked example before shipping. File upload is a documented placeholder, not yet wired up.
- Voice-tag terminology is deliberately different across the two pages: the Media page's "All" voice group displays as "SATB" (changed from "Tutti (full choir)" in v0.13.2, per direct request). The Lyrics page's unfiltered/full-choir state and per-section badges display as "ALL" instead — briefly renamed to "SATB" in the initial v0.13.0 mockup-driven pass, then reverted to "ALL" per the app owner's direct call after seeing it live. Both are confirmed as intentional, not an inconsistency to reconcile.

Originally prepared as a pre-build design plan; updated 2026-07-19 to reflect the app's actual implemented state as of v0.13.3, live at <https://umoja-voices.vercel.app>.